

Formal Runtime Verification for Tool-Using LLM Agents: An Offline Same-Benchmark Study on AgentDojo and STAC

Nikolaos Kekatos
Clone Systems
Larnaca, Cyprus

Stylianios Basagiannis
International Hellenic University
Serres, Greece

Marinelio Chintri
International Hellenic University
Serres, Greece

Alexios Lekidis
University of Thessaly
Larissa, Greece

Tom Nianos
Clone Systems
Larnaca, Cyprus

Ioannis Seitoglou
International Hellenic University
Serres, Greece

Anastasios Temperekidis
Clone Systems
Larnaca, Cyprus

Panagiotis Katsaros
Aristotle University of Thessaloniki
Thessaloniki, Greece

ABSTRACT

Guardrails for tool-using LLM agents are typically implemented as application-specific rules, which makes multi-step, data-dependent policies hard to specify, audit and reuse. We investigate metric first-order temporal logic (MFOTL) as a declarative substrate for such policies, replaying the recorded trajectories that AgentDojo, STAC and R-Judge already ship through the unmodified MonPoly engine. The STAC verdict is agreed by four monitors: our reference implementation, MonPoly, its Isabelle/HOL-verified kernel VeriMon, and the independently developed DejaVu, whose decision procedure differs.

Five generic obligations flag 71.8% of STAC chains and 70.1% of GPT-4o’s *successful* AgentDojo attacks, but also fire on 29.3% of benign runs. Because these corpora rarely record approvals and never record timestamps, the generic obligations reduce to typed-action detection: a benchmark-readiness gap for history-dependent enforcement, which we quantify across the three corpora and answer with a twelve-field enforcement-ready trace schema.

The value of MFOTL becomes clear with *data-parametric* policies: payee-provenance and external-recipient obligations reduce benign firing while preserving most detection. We then show that the provenance signal is itself attackable: one planted structured line defeats the check on 94–99% of the runs it would otherwise flag (deceiving no model, only the monitor), and that binding provenance to the lookup that produced it removes the evasion entirely at no cost in detection or benign firing. A field ablation locates where each capability is gained, and shows that the sound and unsound provenance policies are indistinguishable on clean traces, separating only under attack. Metric policies also flip verdict on identical actions purely by elapsed time.

These results position MFOTL/MonPoly as an auditable substrate for relational and temporal agent policies, complementary to rule-based enforcement, and locate the binding constraint in the corpora rather than the formalism.

CCS CONCEPTS

• **Security and privacy** → **Software and application security**; • **Theory of computation** → *Logic and verification*; • **Computing methodologies** → *Intelligent agents*.

KEYWORDS

runtime verification, LLM agents, agent guardrails, metric first-order temporal logic, provenance integrity, benchmark instrumentation, tool-call traces, CPS/IoT security

ACM Reference Format:

Nikolaos Kekatos, Stylianios Basagiannis, Marinelio Chintri, Alexios Lekidis, Tom Nianos, Ioannis Seitoglou, Anastasios Temperekidis, and Panagiotis Katsaros. 2026. Formal Runtime Verification for Tool-Using LLM Agents: An Offline Same-Benchmark Study on AgentDojo and STAC. In *Joint Workshop on CPS & IoT Security and Privacy (CPSIoTSec ’26)*, November 15, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

The defences shipped with tool-using large language model (LLM) agents are, in practice, mostly *rule-based*. AgentSpec [20], Progent [19] and GuardAgent [21] evaluate predicates on tool calls; the defences benchmarked in AgentDojo [6] (tool filters, prompt spotlighting, prompt repetition) are per-call or per-prompt heuristics. Some of these engines maintain state, and with enough bespoke code a stateful rule can encode a finite-history check, so we do *not* claim rules are incapable of history-dependent enforcement. What they typically lack is a *common declarative semantics* for the multi-step, data-dependent policies that the safety-relevant attacks exploit: each policy is re-encoded per application, in imperative code, hard to audit, reuse, or verify. And those attacks are multi-step: STAC reports attack-success above 90% precisely because the violation is assembled across a chain of individually-benign calls [16].

Monitoring a *trace* against a declarative temporal specification is the defining task of runtime verification. First-order temporal monitors (MonPoly [3], DejaVu [8]) decide metric, quantified past-time properties over event streams with data, which is the exact shape of an agent obligation (“ $\forall$ object o : an irreversible action on o requires a prior approval for o ”). This makes metric first-order

temporal logic (MFOTL) and its monitor MonPoly a natural *declarative substrate* for such policies: quantified over data, with a formal semantics and a reusable, independently-maintained engine, not bespoke per-application code: complementary to rule-based enforcement, not a claim that rules cannot do it.

The empirical question. A paradigm argument is only as good as its evidence: does formal RV actually catch *real* agent attacks, and what does it cost? Answering usually needs a live agent and an LLM. We avoid that: three major benchmarks ship *recorded* trajectories: AgentDojo publishes 36.7k recorded runs (tool-call traces plus its own attack-success and utility labels), STAC publishes 483 recorded attack chains, and R-Judge [23] ships labelled agent records with an IoT subset. We can therefore run a real RV engine over the *same* benchmarks the community uses, entirely offline.

Central finding. The value of formal RV for agent traces depends as much on what the trace *records* as on what the specification language can *express*. On the corpora the community ships, generic temporal obligations collapse into risky-action typing, because approvals, timestamps and state transitions are largely absent; a keyword rule set would reach the same verdicts. Where relational or timing context *is* available, MFOTL expresses more discriminating policies, and those policies then depend on trace content an attacker can write into. Provenance integrity and benchmark instrumentation are therefore not side conditions but prerequisites for history-dependent agent enforcement. That is the argument this paper makes, and each contribution below is a step in it.

Contributions.

- **A declarative RV formulation and same-benchmark evaluation methodology for tool-using agents.** We map recorded agent trajectories into MFOTL event streams and evaluate reusable generic, parametric and metric obligations using independently implemented runtime monitors, scored against each benchmark’s own labels (§3.2).
- **An empirical characterisation of when formal RV adds value.** Generic obligations achieve substantial attack coverage but largely collapse to typed-action detection on existing corpora. Where relational or timing context is available, data-parametric and metric MFOTL policies produce history-dependent verdicts and improve discrimination over action-only checking (§4–§6).
- **The assurance conditions for history-dependent enforcement.** We show that trace provenance is itself an attack surface, demonstrate a lookup-bound provenance mechanism that eliminates the tested poisoning attack, and use a field ablation and a three-benchmark readiness audit to identify the information robust history-dependent enforcement requires. This motivates a twelve-field enforcement-ready trace schema (§5–§6).

Scope. We measure whether an obligation *fires on a recorded trajectory* (detection under post-hoc replay), not live-agent attack success under an in-loop shield. Studying an in-loop shield addresses a *different* question: agent adaptation after enforcement: outside the observational scope of this work.

2 BACKGROUND: AGENT GUARDRAILS AND RV

Rule-based agent guardrails. A per-call guardrail evaluates a propositional predicate on the pending tool call. AgentSpec [20] and Progent [19] give rule languages over the current call; GuardAgent [21] emits guard code from a knowledge base; AgentDojo’s built-in defences [6] filter tools or rewrite prompts; Llama Guard [9] classifies message content. These are effective on their target threats but reason over the current call (or prompt), not the *quantified past* of the trace, and each policy is re-encoded per application.

Benchmarks with recorded traces. AgentDojo [6] evaluates prompt-injection attacks and defences across four suites (banking, slack, travel, workspace) and *ships* the resulting runs: per (model, suite, user task, attack, injection) a JSON with the assistant tool-call trace and the labels security (the injected task was accomplished) and utility (the user task was accomplished). STAC [16] ships 483 recorded multi-turn attack chains (derived from SHADE-Arena [15] and Agent-SafetyBench [24]), each a sequence of benign calls plus a withheld final harmful step. R-Judge [23] ships labelled agent records across domains, including a smart-home / traffic / phone IoT subset. All are replayable offline.

First-order temporal RV. MonPoly [3] monitors MFOTL over timed event streams, quantifying over data; DejaVu [8] is a first-order past-time monitor and VeriMon is MonPoly’s formally verified core: all drop-in for the past-time obligations here. Stream monitors (RTLola [7], TeSSLa [4]) target numeric signals and complement us for quantitative obligations. A *hybrid* deployment can route relational/parametric obligations to MonPoly and purely numeric ones to RTLola/TeSSLa, reusing hybrid-hierarchical RV infrastructure that already runs both families in edge-IoT settings [10–12]; companion work of ours applies mission-level runtime assurance to LLM-assisted swarms over a verification-aware fabric [13], where the traces are generated in simulation, not recorded: the complementary setting to the offline replay studied here; a shield [1] gates the pending call on a violation.

3 OBLIGATIONS AND OFFLINE METHODOLOGY

This section fixes what we monitor and how we measure it. We first state the obligations as MFOTL formulas over a tool-call trace, then describe the pipeline that turns a recorded trajectory into an event stream and the measures reported throughout.

3.1 Five generic obligations in MFOTL

We define five reusable obligations, each of the form “a risky action of a given class requires a matching precondition (approval/consent/confirmation) earlier in the trace,” chosen by action *semantics*, not by any benchmark’s tool names: **O1** irreversible/destructive (delete, cancel, purge), **O2** protection teardown (disable, unlock, add auto-delete rule), **O3** financial commit (transfer, pay, schedule transaction), **O4** external export (send, share, upload), and **O5** actuation/config (set, execute, adjust

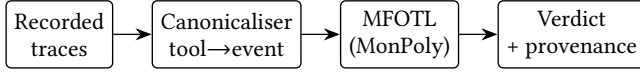


Figure 1: Offline pipeline. Recorded AgentDojo/STAC/R-Judge traces are canonicalised to a typed event stream and checked against MFOTL obligations by MonPoly; verdicts with provenance drive the offline analyses (coverage, BFR, earliest-warning, provenance integrity). No LLM or agent runs.

a device). In MFOTL (O1 shown; $\blacklozenge$ is once-in-the-past):

$$\Box \forall a. (\text{destructive}(a) \rightarrow \blacklozenge \text{approved}(a)).$$

These are the specifications we hand to MonPoly (as their monitored violation form; full formulas and keyword patterns in Appendix A). They are deliberately generic; the study measures how far five reusable formal templates go on real benchmarks, and where precision then demands *parametric* refinement (binding the approval to the specific object, within a window): expressible in MFOTL but not in stateless per-call predicates.

3.2 Replay pipeline and measures

Figure 1 shows the offline pipeline.

Canonicalisation. For each recorded run we build one event stream from the assistant tool calls (AgentDojo: {function, args} per call; STAC: the interaction history plus the withheld final step; R-Judge: the agent-turn actions). The monitor types each tool into an obligation class by semantic keyword patterns and detects preconditions by approval patterns (Appendix A).

Real engine. For STAC we additionally export the canonical stream to a MonPoly log and the obligations to MFOTL, and run the *unmodified* MonPoly binary; its firings are the RV result, not a re-implementation. The CPS/IoT policies of §6.2 are likewise evaluated by MonPoly.

Why MonPoly. The obligations we need are not stateless predicates: they quantify over trace data (payee identity, recipient domain) and bound how far back an enabler counts (consent within 24h, three actuations in 6h). That pair of requirements (first-order quantification plus *metric past*) is exactly safe-range MFOTL, and MonPoly is a maintained implementation of it that runs *unmodified* on an exported log, so our headline firings are not a re-implementation of our own semantics. Among the alternatives surveyed by the runtime verification tools list,¹ DejaVu [8] decides first-order past-time properties with BDDs but offers no metric windows without its timed extension, so the bounded obligations would not transfer directly; TeSSLa [4] and RTLola are stream-based and strong on per-trace numeric computation, but weaker on the first-order joins over unbounded domains (arbitrary payee and recipient strings) that the parametric obligations require. VeriMon, a formally verified sibling of MonPoly, is a drop-in for the same fragment and would be the natural choice where a machine-checked monitor is needed. We therefore treat the engine as interchangeable within the fragment and the *specification* as the contribution.

¹<https://runtime-verification.github.io/tools.html>

Measures. On AgentDojo we use its labels: over attacked runs, *detection on successful attacks* (of security=true runs, the fraction where an obligation fires) and the *benign firing rate* (BFR: of benign runs, the fraction where an obligation fires). BFR is a potential-block proxy for the utility cost (what a shield *would* gate) not a measured task-failure rate, which would require re-running the agent. On STAC (attack-only) we report *final-step detection* (fires on the withheld harmful step); on R-Judge, detection on unsafe records and false-positive rate on safe ones. Confidence intervals are Wilson 95% score intervals.

Benign sampling and parameter choice. The unit is one recorded run. A benign run is one user task with *no* injection (≈ 30 per suite, so 123 benign GPT-4o runs with a parseable trace); an attacked run is a (user task $\times$ injection $\times$ attack) combination (6680). The benign set is thus small, and we report Wilson intervals on every rate (e.g. the 29.3% aggregate BFR has interval [22.0, 37.8]). The keyword patterns are the ordinary English verbs for each class (Appendix A), fixed *before* measuring outcomes and never adjusted; the one threshold, the $\blacklozenge$ window, is the most permissive choice (unbounded past), so a firing is never a tight-window artefact. “Detection” means the obligation fires on the recorded call; a would-be shield gating it assumes prevention. STAC’s content-semantic failure modes (harmful free text) are out of structural scope and flagged, if at all, only via the delivery act.

One correction to the frozen set. The approval list originally contained the token ack, intended for acknowledgement tools. Because matching is on substrings it also matched `invite_user_to_slack` and `remove_user_from_slack` (595 calls in the Slack suite) each of which then acted as a prior approval and suppressed every later obligation in that run. The pattern `authorize` likewise matched the read-only lookup `list_authorized_personnel`. We corrected both after the fact: `ack` is now spelled `acknowledge`, and no read-only accessor (prefix `get_`, `list_`, `read_`, `search_`, `view_`, `fetch_` or `query_`) can constitute an approval event. The correction *raises* the reported rates: AgentDojo detection 61.6% $\rightarrow$ 70.2%, Slack-suite detection 40.3% $\rightarrow$ 63.9%, BFR 31.7% $\rightarrow$ 32.5%, so the pre-correction figures understated the method; they did not inflate it. The same read-only guard then had to be applied to *action typing* as well, because the financial pattern transaction was matching the read-only `get_most_recent_transactions` and `get_scheduled_transactions`: retrieving records is not a value transfer, and the hand-labelled gold set of \$5 marks all twenty read-only accessors NONE. That second correction moved detection to 70.1% and benign firing to 29.3%, and those are the figures reported throughout; it also raised typing precision (micro 0.84 $\rightarrow$ 0.90) and made the payee ablation discriminate at all, since the read-only matches had previously kept the refined financial slot permanently lit. STAC and R-Judge are unaffected: no tool name in either corpus collides. We report the correction instead of silently re-freezing, because it is precisely the class of error a substring canonicaliser invites and precisely what §5 sets out to measure.

4 RESULTS

We report the generic obligations first, then the parametric ones. The contrast between the two matters: the generic layer establishes how much of the recorded attack surface typed-action checking reaches at all, and the parametric layer is where a verdict starts to depend on the trace’s history.

4.1 Generic obligations on recorded trajectories

AgentDojo (same-benchmark, offline). Table 1 replays all recorded GPT-4o runs (no defence). AgentDojo’s own no-defence attack-success is 28.9% (1931/6680). Our RV, applied to the identical runs, fires on 70.1% (95% CI [68.0, 72.1]) of the *successful* attacks: i.e. a shield would have had a flagged call to gate in just over seven in ten of the attacks that actually got through: driven mostly by the financial, export and actuation obligations (teardown never fires: these suites have no teardown tools). The cost is real and we report it plainly: the same generic obligations fire on 29.3% (36/123, Wilson 95% [22.0, 37.8]) of *benign* task runs, because a legitimate task also sends email, moves money and changes config. Detection and BFR rise and fall together under *generic* specs.

Two caveats belong with these numbers. First, they are *typed-action* detection: with no approval events in the corpora, the antecedent of O1–O5 is vacuously satisfied and a firing depends only on the tool’s class, so a keyword rule set would reach the same verdicts. Why that is a property of the corpora and not of MFOTL is the subject of §6; the claims that do rest on the formalism are the parametric obligations of §4.2 and the timed policies of §6.2. Second, a 29.3% BFR is *not a deployable operating point*: BFR counts runs on which some obligation fires, a firing proxy and not a measured utility cost, since we never re-execute the blocked task. At this rate the generic layer is a triage filter for human review, not an enforcement gate.

Across models. The pattern is not GPT-4o-specific: across the 22 base (no-defence) model directories AgentDojo ships, RV detection on successful attacks spans 48.4–91.7% and BFR 16.7–34.8%. AgentDojo’s own built-in defences are not comparable to our offline replay, since they execute the agent *with* the defence in the loop, whereas we never re-run the agent.

Earliest warning: when the obligation fires (prevention timing). A detection is only useful for prevention if it arrives before the harmful step. On the 1931 successful GPT-4o AgentDojo attacks, an obligation fires *before* the final harmful call (*preventive*) in 1012 (52.4%), fires only *at* that call (*last-moment*) in 342 (17.7%), and never fires in 577 (29.9%); where it is preventive the median detection lead is one tool-call (mean 2.19, max 13). On STAC the picture inverts by construction: the corpus *withholds* the final harmful step, so the signal concentrates there: 57/483 (11.8%) preventive, 297 (61.5%) last-moment, 129 (26.7%) no firing. The earliest-warning margin, not a post-hoc residual, is our intervention-timing evidence.

Per suite: where generic obligations break down. The suite breakdown (Appendix B, Table 10) exposes the precision problem concretely. In **banking**, every action is a financial commit, so the generic obligations flag 97.4% of successful attacks and 79.2% of benign runs: near-total recall at near-total BFR. Travel, with few money/export actions, is the opposite (16.5% detection, 11.1%

Table 1: AgentDojo recorded runs (GPT-4o, no defence), replayed offline through our MFOTL obligations. Detection is on AgentDojo’s security=true (successful) attacks; BFR is on its benign (attack=none) runs. The per-obligation counts overlap (one run can trip several obligations) so they sum to 1677, more than the 1354 runs detected; the union, not the sum, is the detection figure. Runs whose tool-call chain is empty or unparseable are excluded (96 of 6899, all in banking). Fuller breakdown in Appendix B, Table 9.

Measure	value
Attacked runs	6680
AgentDojo attack-success (no defence)	1931/6680 = 28.9%
RV detection on <i>successful</i> attacks	1354/1931 = 70.1%
RV fires on any attacked run	2292/6680 = 34.3%
RV BFR on benign runs	36/123 = 29.3%
Wilson 95% interval on BFR	[22.0, 37.8]
<i>by obligation, on the 1354 detected (overlapping)</i>	
destructive without confirmation	307
teardown without approval	0
financial without confirmation	518
export without consent	539
actuation without precondition	313
sum of the five (overlap > union)	1677

Table 2: STAC recorded chains (483) replayed through the same obligations; final-step detection. The MonPoly engine reproduces the 347 firings exactly.

Measure	value
Final-step detection (ref. monitor)	347/483 = 71.8%
confirmed by MonPoly	347 (exact)
structural-intent modes	249/342 = 72.8%
Out-of-template misses	136
Content-semantic (needs semantics)	141 ($\approx 29\%$)

BFR). No single generic threshold is right for all suites; the domain-specific *parametric* refinements (§4.2) are what recover precision.

STAC (recorded chains, MonPoly). Table 2 replays all 483 STAC chains. Our reference monitor flags the withheld harmful action in 347/483 = 71.8% (95% CI [67.7, 75.7]) of chains, and the *unmodified MonPoly engine*, fed the MFOTL obligations and the exported log, fires on *exactly* the same 347: the engine confirms the result. On the structurally-intended failure modes the rate is 72.8%; the content-semantic modes (harmful text, trusting/filtering tool results, $\approx 29\%$ of cases) are out of structural scope and flagged only via the delivery act. 136 chains end in an action outside the five generic classes (spam-rule edits, calendar changes, in-place appends), mapping where new obligations or semantics are needed.

R-Judge: the structural limit on a broad safety benchmark. To probe where structural RV runs out, we replay the *full* R-Judge [23]: 571 records (301 unsafe, 270 safe), a genuine benign set unlike

STAC. The generic obligations fire on 23.6% (71/301) of unsafe records at 16.3% (44/270) FPR. The low detection is expected: most R-Judge unsafe cases are *content-semantic* (privacy leakage, misinformation, unsafe advice) with no structural trace signature, so a rule-based per-call checker fares no better. R-Judge thus *bounds* the reach of structural trace-level RV (formal or rule-based alike) and confirms a semantic aspect is the necessary complement.

4.2 Parametric obligations: relational conditions over trace data

The AgentDojo BFR is not an artefact of RV: it afflicts *any* action-type checker, rule-based or formal, because “sends email” is both the attack and the job. Closing it needs a *precise* condition that quantifies over trace data: allow the transfer only if a matching approval for *that* payee occurred, or the export only if the recipient came from the user’s own directory. Such conditions are expressible in MFOTL and monitorable by MonPoly, and are *not expressible as a stateless per-call predicate*, which cannot refer to the payee’s past provenance. We are careful here: a *stateful* rule engine could of course compute the same condition procedurally, by accumulating seen payees in bespoke code. The advantage we claim is therefore not exclusive computability but *declarative specification*: one formula, with formal temporal/relational semantics, evaluated by a reusable, independently-maintained engine, and hence auditable, in place of per-application imperative state management. We implement two and measure them (Table 3).

Payee provenance (banking). Instead of firing on *any* transfer (generic O3), the parametric obligation fires only on a transfer to a payee never seen as a *structured* recipient or sender in prior account data,

$$\Box \forall p. (\text{transfer}(p) \wedge \neg \blacklozenge \text{known_payee}(p)).$$

On GPT-4o banking runs this cuts the benign firing rate from 44.0% to 24.0% while detection on successful attacks falls only from 79.2% to 70.0%: the injected attacker IBAN is embedded in free text and is never a real counterparty, so it is still flagged, whereas a legitimate transfer to a known payee no longer is.

External recipient (workspace). Symmetrically for export, the parametric obligation fires only on a `send_email` whose recipient never appeared as a structured directory entry (contact/participant/sender field) in a prior tool result,

$$\Box \forall r. (\text{send}(r) \wedge \neg \blacklozenge \text{known_contact}(r)).$$

The injection payload delivers the attacker address inside a free-text email body, not the directory, so it is treated as external; a colleague already in the calendar or address book is not. This cuts the workspace BFR from 15.2% to 10.9% at a detection cost of 42.5% $\rightarrow$ 39.2%.

Discrimination. The positive likelihood ratio LR^+ (detection / BFR) summarises both error rates in one number. Refining the generic obligations ($LR^+ = 2.40$) to the parametric spec raises it on both suites: payee provenance $1.80 \rightarrow 2.92$, external recipient $2.79 \rightarrow 3.61$, so the parametric conjunct separates benign from malicious transfers/exports better than typed-action detection.

Table 3: Parametric vs. generic obligations (GPT-4o), with Wilson 95% intervals. Every row is a *single* obligation on a *single* suite, not a policy-set union, so these rates are not comparable with the pooled run-level figures of Appendix B. The parametric rows fire only when a relational/provenance condition over the trace holds, so their verdict differs from the same call’s typed-action verdict; the generic detection numbers elsewhere are typed-action detection. Benign denominators: 25 banking runs (11/25 and 6/25) and 46 workspace runs (7/46 and 5/46), counting every run with a non-empty message list. On the identical 46 workspace runs the generic *union* of Table 10 fires on 10/46 = 21.7%, against 7/46 = 15.2% for O4 alone: the clearest illustration that the two tables report different policy sets, not different data.

Suite	Spec	detect % [CI]	BFR % [CI]
banking	generic O3	79.2 [75.7, 82.3]	44.0 [26.7, 62.9]
banking	param. payee	70.0 [66.1, 73.6]	24.0 [11.5, 43.4]
workspace	generic O4	42.5 [38.3, 46.8]	15.2 [7.6, 28.2]
workspace	param. ext-recipient	39.2 [35.1, 43.5]	10.9 [4.7, 23.0]

Statistical significance. The parametric obligation is a strict refinement of the generic one, so the paired comparison is concordant except for the discordant cell where the generic fires but the parametric does not. Single-model denominators are too small to be conclusive; pooling benign runs across all AgentDojo model directories (700 banking, 1,376 workspace) makes every discordant run a reduction (banking 75 vs 0, workspace 37 vs 0), with detection largely preserved. The exact McNemar test is then $p = 5.3 \times 10^{-23}$ (banking) and $p = 1.5 \times 10^{-11}$ (workspace).

What depends on temporal/parametric reasoning. We are explicit about the provenance of each number. The generic O1–O5 detection and BFR rates (Tables 1–2) are *typed-action detection*: a firing depends only on the tool’s class, not on trace history, because approvals are almost never present in these corpora. The two parametric rows in Table 3, and every verdict flip in the CPS/IoT study (§6.2), do depend on genuine relational or temporal reasoning: the *same* call is allowed or flagged according to a prior-provenance or prior-state condition, which a *stateless* per-call predicate cannot decide, and which MFOTL states declaratively instead of leaving to bespoke stateful code.

5 THREATS TO VERDICT VALIDITY

A verdict is only meaningful if the two things it rests on can be trusted: the typing that decides an action’s class, and the trace content from which provenance is derived. This section attacks both. The first is an accuracy question we measure against a hand-labelled gold set; the second turns out to be an adversarial question, because the provenance a policy reads is content an attacker can write.

5.1 Canonicalisation accuracy and sensitivity

Detection is only as good as the tool $\rightarrow$ class typing. A single annotator, blind to the typing predictions and to any detection outcome, hand-labelled 54 distinct tool names sampled stratified by call frequency and domain across AgentDojo and STAC. Against this

gold set the keyword typing has micro-averaged precision 0.90, recall 0.84, F1 0.87, and single-primary exact-match 87.0% (47/54); the residual errors are systematic (e.g. `send_channel_message` and `send_direct_message` missed by `export`, `reserve_hotel` and `reserve_restaurant` missed by `financial`) not random, and we report them without tuning (per-class table in Appendix B, Table 11). To bound how much this typing drives the headline numbers, we re-ran the replays under four tool→class mappings (original, corrected, conservative, expanded; Table 12): AgentDojo detection moves in [69.0, 79.9]% and BFR in [26.0, 43.9]%, STAC detection stays in [61.7, 73.1]%; moderately sensitive, with the frozen typing *conservative* on AgentDojo (correcting the mapping raises detection, not lowers it).

5.2 Provenance integrity: the monitor’s trace history is an attack surface

The parametric obligations of §4.2 draw their power from a *provenance* predicate (`known_payee`, `known_contact`), computed over the monitor’s own recorded trace: a payee is trusted iff it appeared earlier as a structured recipient/sender field. That trace is, however, partly attacker-controlled: an injection can plant text that the canonicaliser reads as a structured directory entry, so the history the monitor trusts is itself an attack surface.

Poisoning the provenance signal. We test this directly, injecting one line naming the attacker recipient into each flagged run’s readable context in four framings. A *structured* directory line or a *quoted/forwarded* line flips 94.0% of the 403 flagged banking-payee runs and 99.0% of the 203 flagged workspace-recipient runs to “known” (no longer flagged); a *free-text* mention and a *reformatted* recipient flip 0% (per-framing counts in Appendix B). One planted structured or quoted line therefore defeats the provenance check for 94–99% of the transfers it would otherwise flag: the check trusts *syntactic* structure it cannot authenticate.

A hardened provenance specification. The fix is to make provenance integrity part of the specification. A hardened obligation should require that (i) provenance come from a *trusted tool result* (an account/directory API response), not message text; (ii) a trusted contact *predate the session*; (iii) an approval *bind to the exact payee and amount*, not the action class; and (iv) a *new* payee require explicit confirmation. Conditions (i)–(iv) are themselves MFOTL obligations over a suitably-typed trace (a trust-domain field, a session id, an approval bound to object and amount): exactly the schema current benchmarks lack (§6).

Conditions (ii)–(iv) need fields the corpora do not carry, so they remain design proposals. Condition (i) does not, once stated precisely, and we therefore *evaluate* it. Note first that the obvious reading of (i) is vacuous against this attacker: the injection is itself a forged tool result, and the workspace obligation already restricts accrual to tool messages, yet still flips 99%. What discriminates is *call–answer binding*: a genuine tool result answers a lookup the agent itself issued, while a planted message answers nothing outstanding:

$$\text{known}(p) \leftrightarrow \bullet(\text{lookup}(t) \wedge \bigcirc \text{result}(t, p)).$$

Table 4: Hardened provenance against the poisoning attack of §5.2: flip rate (flagged runs whose verdict reverses to “known”). Detection and benign firing are identical under both predicates, so the hardening costs nothing measurable.

Suite	planted line	naive	hardened
banking (403 flagged)	structured	94.0%	0.0%
	quoted/forwarded	94.0%	0.0%
	free text	0.0%	0.0%
	reformatted	0.0%	0.0%
workspace (203 flagged)	structured	99.0%	0.0%
	quoted/forwarded	99.0%	0.0%
	free text	0.0%	0.0%
	reformatted	0.0%	0.0%

Re-running the same four perturbations against this hardened predicate (Table 4) removes the evasion entirely: structured and quoted framings fall from 94.0% to 0.0% on banking and 99.0% to 0.0% on workspace: at *no* cost: detection (403/576 banking, 203/518 workspace) and benign firing (6/25, 5/46) are unchanged to the run. The fix is not a precision/robustness trade-off; the naive predicate was reading a field it had no warrant to trust. We claim this for the tested attack family only: an attacker who can forge a matching lookup call, or compromise the tool being queried, is outside its reach.

Provenance integrity is therefore a requirement on the trace, not a detail of canonicalisation.

6 BENCHMARK READINESS FOR HISTORY-DEPENDENT AGENT ENFORCEMENT

Our experiments expose a broader evaluation problem. Current agent-security benchmarks record enough to determine whether an *attack succeeded*, but not enough to evaluate many *history-dependent enforcement* policies. We therefore assess the three corpora as enforcement datasets, and quantify the availability of the state, provenance, approval and timing information a runtime monitor quantifies over. This is not a criticism of their design: they were built to measure attack success and agent behaviour, which they do well, but a statement that they are not, as shipped, *enforcement-ready*. The consequence is the phenomenon we reported in §4: with the history absent, sophisticated temporal obligations necessarily collapse into simple action typing.

6.1 The readiness scorecard

A readiness scorecard over the three benchmarks (Table 6) makes this concrete. Explicit *approvals* are almost absent: 0/6899 (0%) AgentDojo runs, 6/483 (1.2%) STAC chains, 2/571 (0.4%) R-Judge records. Recipient/payee identity is recoverable more often (38.8%/58.4%/5.8%) and object identity sometimes (36.4%/34.2%/47.3%), but user-confirmation events are rare (4.6%/16.4%/0%). Above all, *none* of the three carries real time-stamps (only call order), structured state transitions, or reversibility typing. The metric and stateful policies that motivate temporal RV (§6.2, below) thus cannot be evaluated on these corpora as shipped: we authored timed logs to exercise them. As a practical consequence we derive a twelve-field *enforcement-ready* trace

schema, enumerated below, giving the minimum event semantics needed to benchmark relational, provenance-sensitive and timed policies. Recording these fields is cheap at trace-emission time: every one is available to the harness at the moment a tool is called, and it is what would let the next generation of corpora evaluate this class of defence at all. One caveat follows from the census below: fields are necessary but not sufficient. A single-turn benchmark cannot express approval however many columns its log has, so an enforcement-ready corpus is a claim about the *interaction protocol* as much as about the schema.

Why the approvals are absent. The natural objection is that we search for approvals by *tool name* and may be looking in the wrong place. We therefore looked everywhere, and the attrition is itself the result. Two structural facts explain it. AgentDojo is *single-turn*: exactly one user message per run (6899/6899), stating a task up front, after which the agent executes autonomously. An approval is by definition a response to a *proposed* action, which requires a user turn *after* the proposal; the protocol has no such turn, and consistently none of its 68 distinct tools is approval-like. Approvals there are foreclosed by the interaction protocol, not merely unrecorded: no parser can recover them. STAC is *multi-turn* (mean 2.6 user turns, up to five), so approvals *can* occur, and approval language does appear: mining the user text instead of tool names raises the count almost twelvefold, from 6/483 (1.2%) to 70/483 (14.5%) on a loose keyword match and 39/483 (8.1%) on a strict permission-granting match. But only 3/483 (0.6%) occur *before* the harmful step, which is the only position from which an approval could gate anything, because STAC’s attacks work precisely by smuggling the harmful step in without returning to the user. Better extraction therefore does not rescue the situation: it relocates the shortfall from recording to *protocol and position*. An enforcement-ready corpus needs an interaction shape in which approval is expressible, not just a field in which it could be written down.

A control case: τ -bench. If approvals were simply hard to record, no corpus would have them. We therefore ran the same census over the 1,980 recorded trajectories that τ -bench [22] ships, a *task* benchmark whose system prompt is the domain policy and whose policy requires the agent to obtain user confirmation before mutating state. The contrast is stark. τ -bench averages 7.08 user turns per trajectory against AgentDojo’s 1.00 and STAC’s 2.61; 88.7% of its trajectories contain a mutating call; and while a tool-name search again finds *nothing* (0 of its 26 tools is an approval tool, exactly as in AgentDojo), 1039/1980 = 52.5% of trajectories contain explicit permission-granting language in a user turn positioned *before* the mutating call. Every one of those 1039 answers a preceding assistant turn that made no tool call, i.e. they are confirmations of a proposal rather than instructions. Against STAC’s 0.6% that is a factor of eighty.

Two conclusions follow. The approval shortfall is a property of *attack* corpora rather than of agent traces in general, so it is a fixable instrumentation gap and not an intrinsic limit; and the fix is already partly available, since on a corpus like τ -bench the antecedent of O1–O5 would be genuinely satisfiable and the generic obligations would not collapse to action typing. We do not report detection on τ -bench because it carries no attack labels: it measures task success, not attack success, which is why the detection results

Table 5: Trace-field ablation on AgentDojo banking. Levels are cumulative; “poison flip” is the structured-line evasion rate of §5.2. R1 and R2 differ only under attack.

Level	policy class	detect	BFR	flip
R0 action only	typed-action	79.2%	44.0%	n/a
R1 + recipient id	recipient-bound	70.0%	24.0%	94%
R2 + provenance	poison-resistant	70.0%	24.0%	0%
R3 + approval id	approval-bound	unevaluable (0/1545 runs)		
R4 + time, state	metric	unevaluable (0 timestamps)		

Table 6: Benchmark readiness for history-dependent enforcement: fraction of runs/records exposing each field. Real timestamps, structured state transitions, and reversibility typing are absent from all three (call-order only).

Field	AgentDojo	STAC	R-Judge
explicit approval	0%	1.2%	0.2%
recipient/payee id	38.8%	58.4%	5.8%
object id	36.4%	34.2%	47.3%
user confirmation	4.6%	16.4%	0%
real timestamps	none	none	none
structured state	none	none	none
reversibility typing	none	none	none

here use AgentDojo and STAC. The two are complementary, and a corpus with τ -bench’s interaction protocol and AgentDojo’s attack labels is, on this evidence, the single most useful thing the community could build.

The contribution of each field. A scorecard says which fields are missing; it does not say what capability their absence costs. We therefore ablate the trace itself, masking fields cumulatively on AgentDojo banking and asking, at each level, which policy class remains evaluable and how well it discriminates (Table 5). Two results stand out. First, the levels the corpora *do* support are exactly the ones that buy little: recipient identity lifts LR⁺ from 1.80 to 2.92, and then the approval- and timing-dependent levels are unevaluable: *zero* approval-typed calls and *zero* timestamped messages across 1545 runs. Second, and less obviously, R1 and R2 are *observationally identical* on clean traces: same detection, same benign firing, same LR⁺. They separate only under attack, at 94% versus 0% evasion. Field availability alone therefore does not tell you whether a provenance policy is sound; the binding does, and its value is invisible to any evaluation that does not include an adversary. This is the sharper form of the readiness argument: a corpus can expose every field a policy names and still be unable to distinguish a sound policy from an unsound one.

A minimal trace schema. We therefore propose a minimal enforcement-ready schema: twelve fields per event: timestamp, actor, tool, action, object, recipient, source, approval_id, session_id, state_before, state_after, trust_domain. Current benchmarks supply the middle group (tool/action, partially object/recipient) but none of wall-clock timestamp, approval_id, session_id, the state pair, or trust_domain: the fields the timed policies (§6.2) and the hardened provenance spec (§5.2) require.

Table 7: Timed (metric MFOTL) policies evaluated verbatim by MonPoly. For each, the *same* action is allowed or a violation purely by elapsed time; • marks the timepoints where MonPoly fires. Full sig/log/formula in Appendix A.

Policy (window)	action, timing	MonPoly
T1 consent (24 h)	export, consent 12 h ago	pass
T1 consent (24 h)	export, consent 48 h ago	• violation
T1 consent (24 h)	export, re-consent 10 h ago	pass
T2 rate ($\geq 3/6$ h)	3rd actuation within 6 h	• violation
T2 rate ($\geq 3/6$ h)	actuators spaced > 6 h apart	pass
T3 precede (8 h)	execute, auth 6 h ago	pass
T3 precede (8 h)	execute, auth 40 h ago	• violation
T3 precede (8 h)	execute, re-auth 4 h ago	pass

Cheap to emit but absent everywhere, they are the concrete obstacle to evaluating history-dependent agent enforcement.

6.2 Timed CPS/IoT policies: evaluability under the proposed schema

This is a proof of *applicability* of MFOTL/MonPoly to CPS/IoT policies, not a broad security validation on CPS/IoT deployments. Agentic components are already being proposed for such settings, together with tamper-evident evidence trails [14], which is what makes the trace-integrity question of §5 concrete rather than hypothetical. We give three evidence points, all evaluated verbatim by MonPoly: *timed* (metric) actuation policies, a multi-instance building trace, and a real (small) IoT subset.

Timed policies whose verdict depends on elapsed time. A reviewer of an earlier version noted the generic obligations exercise little temporal complexity: on these corpora an unbounded $\blacklozenge$ reduces to typed-action detection. We therefore author three *metric* obligations whose verdict on the *identical* action changes purely with the clock (signatures, logs and formulas in Appendix A; Table 7). (T1) *Bounded-window consent*: an export requires a consent within the last $W=24$ h, $\text{export}(o) \wedge \neg(\text{ONCE}[0, 24] \text{ consent}(o))$: MonPoly fires only when the consent is stale. (T2) *Sliding-window rate limit*: ≥ 3 actuators on a device within $W=6$ h (metric CNT over $\text{ONCE}[0, 6]$): fires on a burst, silent once calls are spaced. (T3) *Timed precedence*: an execute must be preceded within $W=8$ h by an authorisation: fires only when the authorisation has expired. Each is exactly the kind of “property that contains time” a per-call classifier, and an unbounded past-time check, cannot express.

The traces behind Table 7 are likewise authored, not recorded: the point they establish is that *elapsed time alone* flips MonPoly’s verdict on an otherwise identical action, which is a property of the specification language, not evidence of field accuracy.

Multi-instance building actuation. To show the effect is not a single hand-picked pair, we author a 20-event smart-building trace (13 policy-relevant actions) exercising three past-time policies with multiple compliant and violating instances each (envelope-plus-maintenance-mode, a timed actuator dependency, and badge-authorised access; formulas in Appendix A). MonPoly flags exactly the 6 history-violating instances and clears all 7 compliant ones; one violation is a purely *timed* miss: an enabling

damper_open that fell 4 h before a 3 h window, so the same compressor start is a violation only by timing.

R-Judge IoT (real data, small: cannot be enlarged). The R-Judge IoT subset (smart home, traffic dispatch, phone) is *all the labelled IoT agent records the benchmark ships*: 30 records (19 unsafe, 11 safe), so we report it as illustrative. The generic obligations yield 31.6% detection (6/19) and 27.3% FPR (3/11) with wide Wilson intervals. It does contain relational cases: in one unsafe record the agent grants a dog-walker *permanent* smart-lock access without consent. Encoding it under a parametric consent obligation $\Box \forall g, p. (\text{grant}(g, p) \wedge p=1 \wedge \neg \blacklozenge \text{consent}())$, MonPoly fires on the permanent grant and clears once a prior consent is present: a verdict flip on real IoT data the generic “grant” typing (which never fires here) cannot make.

7 DISCUSSION AND CONCLUSION

Positioning. MFOTL and MonPoly have already been applied to agent policy enforcement: Maris [5] expresses privacy policies in MFOTL and calls MonPoly online, from inside the agent runtime, on three authored applications. Our question is a different one, and the difference is the corpus rather than the formalism: what is measurable *offline*, on the attack corpora the community already ships, and what those corpora cannot support. What formal RV adds in that setting is the combination: trace-level scope, formal temporal semantics, data-parametric provenance and offline evaluability in *one* declarative substrate. The comparison is with what the cited systems were shown to do, not with what they could in principle express. AgentDojo [6], AgentBench [17] and ToolEmu [18] *evaluate* agents, and the guardrails they benchmark are, as reported, per-call or rule-based. AgentSpec [20] has a formal rule semantics and some history-aware rules ($\circ$), so a determined encoding could reach further than Table 8 shows; that table records evaluated configurations, not impossibility.

Limitations. We measure firing on recorded trajectories, not live-agent prevention; an in-loop shield addresses a different question (agent adaptation after enforcement) outside our scope. Generic specs have a high BFR (29.3%), and the parametric refinements that reduce it are evaluated for banking and workspace only. STAC lacks preconditions (6/483 chains by tool name; §6), so its generic obligations reduce to typed-action detection; the metric power of MFOTL is exercised in the CPS/IoT study instead. Keyword typing is coarse, and a semantic residual ($\approx 29\%$ of STAC, more of R-Judge) needs a content checker structural RV does not provide. Finally, the timed policies run the real MonPoly engine but on authored logs that isolate the timing dependence: establishing expressiveness, not field prevalence.

Conclusion. Offline on real agent-attack corpora, MonPoly flags 71.8% of STAC chains and detects 70.1% of GPT-4o’s successful AgentDojo attacks at 29.3% BFR: numbers that mostly reflect risky-action *typing*, since these corpora rarely contain the preconditions temporal logic is for. Where data-parametric and timed reasoning applies, MFOTL states and MonPoly checks conditions a stateless per-call predicate cannot. Four completed results anchor the security case: a concrete earliest-warning margin (preventive on 52.4% of successful AgentDojo attacks, median lead

one tool-call); the provenance predicate behind the parametric precision is itself an attack surface, flipped 94–99% by one planted structured line, which binding provenance to the issuing lookup closes completely on the tested attack family, at no cost in detection or benign firing; a field ablation showing that the sound and unsound variants are indistinguishable except under attack; and a readiness scorecard showing current benchmarks lack the timestamps, structured state, reversibility typing and approvals history-dependent enforcement needs. That last result is the one we would most want carried forward: widely used agent-security corpora support attack-level evaluation but systematically omit the event semantics required to test history-dependent enforcement. It explains why generic temporal policies collapse to action typing on current corpora, and it motivates the enforcement-ready trace schema of §6 as the concrete thing a future benchmark could adopt. MFOTL/MonPoly is thus a reusable, auditable substrate for quantified, timed agent-trace policies, complementary to rule-based enforcement and measurable offline on the corpora the community already ships.

REFERENCES

- [1] Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. 2018. Safe Reinforcement Learning via Shielding. In *Proc. 32nd AAAI Conf. on Artificial Intelligence (AAAI)*.
- [2] David Basin, Thibault Dardinier, Nico Hauser, Lukas Heimes, Jonathan Julián Huerta y Munive, Nicolas Kaletsch, Srđan Krstić, Emanuele Marsicano, Martin Raszyk, Joshua Schneider, Dawit Legesse Tirore, Dmitriy Traytel, and Sheila Zingg. 2022. VeriMon: A Formally Verified Monitoring Tool. In *Proc. 19th Int. Colloquium on Theoretical Aspects of Computing (ICTAC) (LNCS, Vol. 13572)*. Springer, 1–6.
- [3] David Basin, Matúš Harvan, Felix Klaedtke, and Eugen Zălinescu. 2011. MonPoly: Monitoring Usage-Control Policies. In *Proc. 2nd Int. Conf. Runtime Verification (RV)*.
- [4] Lukas Convent, Sebastian Hungerecker, Martin Leucker, Torben Scheffel, Malte Schmitz, and Daniel Thoma. 2018. TeSSLa: Temporal Stream-Based Specification Language. In *Proc. 21st Brazilian Symp. Formal Methods (SBMF)*.
- [5] Jian Cui, Zichuan Li, Chi Wang, Luyi Xing, and Xiaojing Liao. 2026. Maris: A Formally Verifiable Privacy Policy Enforcement Paradigm for Multi-Agent Collaboration Systems. *arXiv:2505.04799 [cs.CR]*
- [6] Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *NeurIPS Datasets and Benchmarks Track*.
- [7] Peter Faymonville, Bernd Finkbeiner, Malte Schledjewski, Maximilian Schwenger, Marvin Stenger, Leander Tentrup, and Hazem Torfah. 2019. StreamLAB: Stream-Based Monitoring of Cyber-Physical Systems. In *Proc. 31st Int. Conf. Computer Aided Verification (CAV)*.
- [8] Klaus Havelund, Doron Peled, and Dogan Ulus. 2018. DejaVu: A Monitoring Tool for First-Order Temporal Logic. In *Proc. 3rd Workshop on Monitoring and Testing of Cyber-Physical Systems (MT-CPS)*.
- [9] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yunying Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabba. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2312.06674* (2023).
- [10] Nikolaos Kekatos, Marinello Chintiri, Panagiotis Katsaros, Alexios Lekidis, Tom Nianos, Ioannis Seitoglou, and Stylianos Basagiannis. 2026. Hierarchical Security Monitoring for Edge-IoT: A Formal Methods Approach. In *Proc. IEEE Int. Conf. on Cyber Security and Resilience (CSR)*. IEEE.
- [11] Nikolaos Kekatos, Marinello Chintiri, Panagiotis Katsaros, Alexios Lekidis, Tom Nianos, Ioannis Seitoglou, Anastasios Temperekidis, and Stylianos Basagiannis. 2026. A Resilient Runtime-Verification Fabric for Security Monitoring of Critical Edge-IoT Infrastructure. In *Proc. 21st Int. Conf. on Critical Information Infrastructures Security (CRITIS) (LNCS)*. Springer.
- [12] Nikolaos Kekatos, Marinello Chintiri, Panagiotis Katsaros, Alexios Lekidis, Tom Nianos, Ioannis Seitoglou, Anastasios Temperekidis, and Stylianos Basagiannis. 2027. Hybrid Hierarchical Runtime Verification for Edge-IoT Security: Combining MonPoly and RTola. In *Availability, Reliability and Security. ARES 2026 EU Projects Symposium Workshops (LNCS, Vol. 16903)*. Springer, Cham, 222–240. https://doi.org/10.1007/978-3-032-37218-5_14
- [13] Nikolaos Kekatos, Panagiotis Katsaros, Alexios Lekidis, Theodoros Nestoridis, and Tom Nianos. 2026. Mission-Level Runtime Assurance for LLM-Assisted
- ISR Swarms over a Verification-Aware Fabric. *arXiv:2607.23532 [cs.CR]*
- [14] Nikolaos Kekatos, Georgios Koutidis, Konstantinos Antonakopoulos, Stylianos Basagiannis, Sokratis Katsikas, Georgios Kavallieratos, Alexios Lekidis, Tom Nianos, and Elpiniki Papageorgiou. 2026. RESILAGENT: A Three-Pillar Architecture for Cyber-Physical Resilience in Electrical Power and Energy Systems. In *Proc. IEEE Int. Conf. on Cyber Security and Resilience (CSR), EPES Track*. IEEE.
- [15] Jonathan Kutasov, Yuqi Sun, Paul Colognese, Teun van der Weij, Abhay Sheshadri, et al. 2025. SHADE-Arena: Evaluating Sabotage and Monitoring in LLM Agents. *arXiv preprint arXiv:2506.15740* (2025).
- [16] Jing-Jing Li, Jianfeng He, Chao Shang, Devang Kulshreshtha, Xun Xian, Yi Zhang, Hang Su, Sandesh Swamy, and Yanjun Qi. 2025. STAC: When Innocent Tools Form Dangerous Chains to Jailbreak LLM Agents. *arXiv preprint arXiv:2509.25624* (2025).
- [17] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. AgentBench: Evaluating LLMs as Agents. In *Int. Conf. on Learning Representations (ICLR)*.
- [18] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *Int. Conf. on Learning Representations (ICLR)*.
- [19] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. *arXiv preprint arXiv:2504.11703* (2025).
- [20] Haoyu Wang, Christopher M. Piskitt, and Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. In *Proc. 48th IEEE/ACM Int. Conf. on Software Engineering (ICSE)*.
- [21] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. 2024. GuardAgent: Safeguard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning. *arXiv preprint arXiv:2406.09187* (2024).
- [22] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *Proc. Int. Conf. on Learning Representations (ICLR)*.
- [23] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. In *Findings of the Association for Computational Linguistics: EMNLP*.
- [24] Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. 2024. Agent-SafetyBench: Evaluating the Safety of LLM Agents. *arXiv preprint arXiv:2412.14470* (2024).

A REPRODUCIBILITY

This appendix is excluded from the page limit. All scripts are pure-stlib Python in `agent-rv-impl/`; the datasets are the public releases below.

Datasets. STAC (*arXiv:2509.25624*, `amazon-science/Multi-TurnAgentAttack`, 483 chains, file `STAC_benchmark_data.json`); AgentDojo (`ethz-spylab/agentdojo`, recorded runs/`<model>/<suite>/...`, $\approx 36.7k$ runs; we use `gpt-4o-2024-05-13` in detail and, for the cross-model analysis, the 22 base (no-defence) model directories the release ships, plus their defence variants where pooled: 28 directories with a banking suite and 29 with a workspace suite); R-Judge (`Lordog/R-Judge`, `data/`, 571 records; the IoT subset `data/IoT/{household,trafficdispatch,phone_iot}.json` is all 30 IoT records the benchmark ships and cannot be enlarged offline).

Keyword patterns (typing). A tool matches a class iff its lower-cased name contains any listed substring. **DESTRUCTIVE:** delete, remove, cancel, clear, wipe, drop, purge, erase, revoke, uninstall. **TEARDOWN:** autodelete, disable, stop, unlock, deactivate, turn_off, bypass, whitelist, allowlist, add_user_autodelete. **FINANCIAL:** send_money, transaction, transfer, payment, pay_, book_, buy, purchase, zelle, venmo, wire, invoice, checkout. **EXPORT:** send_email, share, export, upload, post_, publish, forward, sendmessage, send_message, send_sms, tweet. **ACTUATION:** set_,

adjust, trafficleight, unlock_door, execute, terminal, medication, open_, close_, activate, reboot, restart, deploy, write_, update_, configure, install. *APPROVAL* (precondition): confirm, approve, consent, authorize, authorise, permission, verify, grant, two_factor, 2fa, otp, request_approval, acknowledge, sign_off, double_check. No tool whose name begins get_, list_, read_, search_, view_, fetch_ or query_ counts as an approval event or as a typed action, since a read-only accessor is neither a grant of permission nor a commit (§3.2). This is the corrected set used for every number reported here.

Argument normalisation. Tool arguments are parsed from JSON to a dict (malformed strings kept under a raw key). The known_payee and known_contact predicates are *not* hand-curated: they are emitted by the same scripted canonicaliser via a fixed regular expression over structured recipient/sender fields, and the regexes were frozen *before* we inspected any attacker IBAN/address or any security/utility outcome. For the payee obligation a recipient is a *known payee* only if it matched $(?:recipient|sender):\backslash s*([A-Z]{2}[0-9A-Z]{6,})$ in prior message text (a structured IBAN field, not free text). For the external-recipient obligation an address is a *known contact* only if it appeared under a directory key $(?:from|sender|recipients?|participants?|email|shared_with|owner):$ in a prior *tool* result. So a payee/contact becomes “known” purely by the scripted rule, independent of whether it is later an attack address.

Approval/consent recognition. An approval precedes step i iff some call at $j < i$ (same trace, same STAC case) matches the APPROVAL patterns. An obligation fires on step i iff the tool matches its class patterns *and* no approval precedes i .

Timestamps and multiple firings. Each canonical event becomes one MonPoly timepoint $@t$; for STAC the class and approval predicates are parameterised by the case id so ONCE only matches within a case. Only timepoints that carry at least one typed predicate are emitted: a canonicalised step whose tool matches *no* obligation class (a read-only or out-of-template action) produces no event, so the log holds 357 predicate-bearing timepoints across the 483 cases, and cases whose final step matches no class emit nothing and cannot fire (counted as a miss). Because the STAC obligations use an unbounded ONCE[0, 10⁶], dropping predicate-free timepoints is sound: an unbounded past-time operator is insensitive to the number of intervening empty timepoints, only to whether a matching predicate occurred earlier in the case. For the *timed* policies of §6.2 this is different: there the integer timestamp is the event’s clock time (in hours), timepoints are spaced by real elapsed time, and the metric windows (ONCE[0, W]) depend on those gaps, so we keep the actual times, not a dense index. Per-obligation counts are non-exclusive; a case/run counts as detected if ≥ 1 obligation fires.

Full MFOTL obligations (monitored violation form). With $\blacklozenge$ once-in-the-past:

- O1 : destructive(a) $\wedge \neg \blacklozenge$ approved(a)
- O2 : teardown(a) $\wedge \neg \blacklozenge$ approved(a)
- O3 : financial(a) $\wedge \neg \blacklozenge$ approved(a)
- O4 : export(a) $\wedge \neg \blacklozenge$ approved(a)
- O5 : actuation(a) $\wedge \neg \blacklozenge$ approved(a)

The STAC export uses the disjunction handed verbatim to MonPoly: (destructive(c) OR teardown(c) OR financial(c) OR export(c) OR actuation(c)) AND NOT (ONCE[0,1000000] approval(c)). The two parametric obligations are $\Box \forall p.(\text{transfer}(p) \wedge \neg \blacklozenge \text{known_payee}(p))$ and $\Box \forall r.(\text{send}(r) \wedge \neg \blacklozenge \text{known_contact}(r))$. The multi-instance building policies, run verbatim by MonPoly, are:

```
P1: setpoint(z,v) AND v > 28 AND NOT ((NOT maint_off())
SINCE maint_on())
P2: compressor_start(z) AND NOT (ONCE[0,3]
damper_open(z))
P3: door_unlock(d) AND NOT (ONCE[0,3] badge_auth(d))
lock: grant_access(g,p) AND p = 1 AND NOT (ONCE
consent())
```

The 20-event building log yields firings at timepoints {4, 5, 9, 14, 19, 22} (6 violations; the P2 firing at $t=14$ is the timed miss, a damper_open at $t=10$ outside the 3-unit window) and no firing on the 7 compliant instances.

Timed (metric) policies of §6.2. These use hour-valued timestamps; formula, signature and log are run verbatim by MonPoly:

```
T1: export(o) AND NOT (ONCE[0,24] consent(o))
log: @0 consent(1) @12 export(1) @48 export(1) @50
consent(1) @60 export(1)
⇒ fires only @48 (consent 48h stale)
T2: (c <- CNT s; d (ONCE[0,6] actuate(d,s))) AND c >=
3
log: @0 actuate(7,1) @2 actuate(7,2) @5 actuate(7,3)
@30.. @33.. @40..
⇒ fires only @5 (3rd actuation within 6h; spaced
calls never fire)
T3: execute(x) AND NOT (ONCE[0,8] request(x))
log: @0 request(5) @6 execute(5) @40 execute(5) @44
request(5) @48 execute(5)
⇒ fires only @40 (authorisation 40h old)
```

Each timed verdict flips on the *same* action purely by elapsed time; T2 uses MonPoly’s metric CNT aggregation over a sliding [0, 6] window.

MonPoly version and invocation. The engine is MonPoly, version string MonPoly (development build) (from monpoly -version), at /usr/local/bin/monpoly inside Docker image infsec/monpoly (image id sha256:82cde1a23cd3). It is invoked as monpoly -sig <s>.sig -formula <f>.mfotl -log <l>.log; all formulas above pass monpoly -check as monitorable. On the STAC export it fires on exactly 347 timepoints, matching the reference monitor. Because both consume the same canonical log, that agreement checks our implementation rather than the specification, so we add a genuinely independent engine. **DejaVu** [8] decides first-order past-time properties by a different procedure (BDDs over quantified past-time LTL). The STAC obligation is untimed past-time first order, so it transfers to DejaVu’s QTL without weakening; on the same event stream DejaVu reports exactly 347 violations. We additionally run **VeriMon** [2], MonPoly’s formally verified kernel (monpoly -verified), whose monitoring algorithm is machine-checked in Isabelle/HOL: also exactly 347. So the obligation is agreed by four monitors: our reference implementation, MonPoly’s standard kernel, its verified kernel, and an independently developed BDD-based engine: of which the last two make the result something stronger than agreement between MonPoly and a checker we wrote ourselves: one is a different decision procedure, and one is proved correct.

Table 10: Per-suite replay (GPT-4o, no defence): AgentDojo attack-success, RV detection on successful attacks, and RV BFR on benign runs. Detection and BFR here are *policy-set unions* over all five generic obligations, so they exceed the single-obligation rates of Table 3 on the same suite: workspace BFR is 10/46 as a union against 7/46 for O4 alone. This table also counts only runs with a non-empty tool-call chain (96 banking runs are excluded on that ground, one of them benign), whereas Table 3 counts every run with a non-empty message list: the banking gap therefore has two causes at once. Under the message-list filter the generic union fires on $19/25 = 76.0\%$ of benign banking runs against $19/24 = 79.2\%$ here, the same numerator over the one extra run; Table 3’s $11/25$ differs from both because it is a single obligation, not the union.

Suite	ASR	RV detect	BFR
banking	40.1%	97.4%	79.2%
slack	60.9%	63.9%	15.4%
travel	9.3%	16.5%	11.1%
workspace	19.9%	62.9%	21.7%

Artifact availability. All evaluation scripts are pure-stdlib Python under `agent-rv-impl/`; together with the MonPoly signature/log/formula files for the STAC, building, IoT and timed studies they will be released as supplementary material (and the public repository <https://github.com/nikos-kekatos/formal-rv-tool-using-llm-agents>), so the full pipeline is reproducible against the public dataset releases named above.

AgentDojo selection and unflagged successful-attack fraction. Runs are read by recursive glob of `runs/<model>/**/*.*.json`. A run is *attacked* iff `attack_type` is set and `≠none` and `injection_task_id` is present; *benign* iff neither is present; runs with an empty/unparseable tool-call chain are skipped (96 for GPT-4o $\Rightarrow$ 6680 attacked, 123 benign). `security==True` marks a successful attack, `utility==True` a completed task. For reference we report the *unflagged successful-attack fraction*, ($\text{successful} \wedge \neg \text{RV-fired}$)/attacked = $(1931 - 1354)/6680 = 8.6\%$; this is a descriptive count of successful attacks the monitor did not flag, *not* a measured attack-success rate under an in-loop shield (which would require re-running the agent).

B EXTENDED RESULTS AND ROBUSTNESS

Table 8: Paradigm comparison, *per each system’s evaluated configuration* (not a fundamental-capability claim). TL = trace-level; FS = formal temporal semantics; PAR = parametric (quantifies over trace data); OFF = evaluable offline on recorded runs. $\circ$ = partial / achievable with bespoke stateful code.

	TL	FS	PAR	OFF
AgentDojo defences [6]	–	–	–	✓
AgentSpec [20]	$\circ$	✓	$\circ$	–
Progent [19]	$\circ$	–	–	–
GuardAgent [21]	–	–	–	–
STAC [16] (benchmark)	✓	–	–	✓
Formal RV / MonPoly (ours)	✓	✓	✓	✓

Per-obligation ablation (AgentDojo, GPT-4o). Table 9 breaks the union result down by obligation, with each obligation’s positive likelihood ratio LR^+ (detection / BFR). Destructive is the single most discriminating obligation ($LR^+ = 3.91$), ahead of export (2.64) and teardown never fires for GPT-4o (these suites have no protection-teardown tools).

Table 9: Per-obligation detection on successful attacks and BFR (AgentDojo, GPT-4o), with unique flagged successful attacks and $LR^+ = \text{detection/BFR}$. Union is the headline any-obligation result.

Obligation	detect %	BFR %	unique	LR^+
O1 destructive	15.9	4.1	259	3.91
O2 teardown	0	0	0	–
O3 financial	26.8	13.0	243	2.06
O4 export	27.9	10.6	491	2.64
O5 actuation	16.2	6.5	38	2.49
union (any)	70.1	29.3	–	2.40

Cross-model sign test (parametric payee, banking). Beyond the pooled McNemar test (§4.2), we check the direction of the parametric effect per model. Across the 22 base (no-defence) model directories with a banking suite, the parametric payee obligation *decreases* benign firing in 20, leaves it unchanged in 2, and increases it in 0; the median per-directory reduction is 8.0 percentage points (IQR [4, 17], min 0, max 24). An exact two-sided sign test on the 20-vs-0 split gives $p = 1.9 \times 10^{-6}$: the reduction direction is consistent across models, not an artefact of pooling. Over the same 22 directories, generic detection ranges 48.4–91.7% and BFR 16.7–34.8%.

Table 11: Canonicalisation typing vs. 54 hand-labelled tools. Per-class multi-label precision/recall (does the typing include the gold class?), micro-average, and single-primary exact-match.

Class	P	R	(tp/fp/fn)
O1 destructive	0.78	1.00	7/2/0
O2 teardown	1.00	1.00	3/0/0
O3 financial	1.00	0.78	7/2/2
O4 export	1.00	0.67	4/0/2
O5 actuation	0.86	0.86	6/1/1
micro-average	0.90	0.84	F1 0.87
single-primary exact-match:	47/54 = 87.0%		

Canonicalisation sensitivity. Table 12 re-runs the AgentDojo and STAC replays under four tool $\rightarrow$ class mappings. Correcting the known typing errors raises detection and BFR together (the frozen keyword typing is conservative); the conservative mapping lowers AgentDojo detection to 69.0% and BFR to 26.0%, and STAC further.

Table 12: Sensitivity of the headline rates to the tool→class mapping. AD = AgentDojo (GPT-4o); STAC detection is final-step.

Mapping	AD detect %	AD BFR %	STAC detect %
original	70.1	29.3	71.8
corrected	79.9	43.9	73.1
conservative	69.0	26.0	61.7
expanded	79.9	43.9	73.1

MonPoly runtime and scale-up soundness. Table 13 reports throughput and peak resident memory for the STAC log at $1\times$, $10\times$ and $50\times$ replication and for an AgentDojo-derived log. Throughput is dominated by start-up on short logs, rising from 52–179k events/s on the 357-timepoint STAC log to 0.43–0.45M at $10\times$ – $50\times$ and 0.8M on the AgentDojo export, with flat 8–14MB RSS; at $\times 1$ the parametric formula is about three times slower than the generic one (52k vs 155k), a gap that closes as the log grows. Replicating the STAC log $10\times$ yields exactly $3470 = 10 \times 347$ firings, which checks determinism and linear scaling rather than soundness.

Table 13: MonPoly throughput and peak RSS. STAC $\times k$ is the STAC log replicated k times; firings scale exactly ($10 \times 347 = 3470$).

Log	throughput (events/s)	peak RSS
STAC $\times 1$	52k–179k	8–14 MB
STAC $\times 10$	452k	8–14 MB
STAC $\times 50$	429k	8–14 MB
AgentDojo-derived	≈ 0.8 M	8–14 MB

Synthetic corpus (AgentTrace-T) and metamorphic tests. To exercise the specifications where the public corpora lack the needed fields, we generated *AgentTrace-T*: 1400 traces, 200 per policy across P1–P3, T1–T3 and the IoT-lock policy, in three generated kinds (core, and boundary cases on each side of the window), each labelled compliant or violating, roughly 100 of each per policy. MonPoly achieves 100% verdict accuracy, 100% parameter-binding accuracy, and 100% on the boundary cases. A metamorphic test suite then applies six semantics-preserving or semantics-changing transformations: moving an enabler across the window boundary, changing an id, reordering, inserting an irrelevant event (verdict must not change), duplicating an event (must not change), and boundary shifts, and MonPoly gives the expected verdict on 780/780 (100%). These perfect scores must be read for what they are: AgentTrace-T was authored *against* the same policies MonPoly then checks, so 100% measures whether the metric and first-order operators behave as specified (an implementation and semantics check) and not whether the policies catch attacks in the

field. No field corpus is involved, and none of the headline detection numbers rests on this corpus. We report it because the public corpora lack the timestamps and approval events these operators need (§6), so there is otherwise no way to exercise them at all.

Combined parametric ablation. The rates in this paragraph are *run-level policy-set* rates over pooled runs: a run counts as flagged if *any* obligation in the set fires on it. Applying *both* parametric obligations together, the banking+workspace pooled rates move from the union of all five generic obligations at 80.6% detection / 40.8% BFR to 79.6% / 32.4%, and the four-suite overall rates from the same generic union at 69.9% / 29.0% to both-parametric 69.4% / 24.2%. These pooled figures are unions over the full generic set and are therefore *not* comparable with the single-obligation rows of Table 3: a run that O3 alone misses can still be caught by another generic obligation, so the union can exceed any individual obligation’s rate. Run-level BFR barely moves because benign runs also trip *other* generic obligations; the parametric win is at the *action slot*: over the pooled banking+workspace benign runs the export slot BFR falls 9.9% $\rightarrow$ 7.0% and the send_money slot 22.5% $\rightarrow$ 16.9%, which is where the provenance conjunct applies. (Before the read-only correction of §3.2 the send_money slot appeared not to move at all, because read-only accessors matching the financial pattern kept the slot lit regardless of the refinement.)

Provenance poisoning (detail). The poisoning experiment (§5.2) injects, per flagged run, one line naming the attacker recipient in four framings and re-evaluates the parametric obligation. Banking payee (403 flagged successful-attack runs): structured 379/403 = 94.0% flip, quoted/forwarded 94.0%, free-text 0%, reformatted-IBAN 0%. Workspace recipient (203 flagged): structured 201/203 = 99.0%, quoted 99.0%, free-text 0%, reformatted 0%.

Scripts. stac_rv.py, stac_to_monpoly.py, agentdojo_rv.py, agentdojo_scenarios.py, agentdojo_param.py (payee), agentdojo_param2.py (external recipient), agentdojo_param_pooled.py (pooled cross-model McNemar: produces the headline p -values), cross_model.py (per-directory cross-model sign test), first_firing.py (earliest-warning), combined_ablation.py and per_oblig_ablation.py (ablations), risk_enrichment.py (LR⁺ and balanced accuracy), canon_sensitivity.py (four-mapping sensitivity), perturb_provenance.py (poisoning), readiness_scorecard.py (benchmark scorecard), corpus_gen.py (AgentTrace-T), metamorphic.py (metamorphic tests), mp_time.py (MonPoly runtime), canon_eval.py, wilson_ci.py, mcnemar.py (single-model paired-benign, the inconclusive per-model result), rjudge_rv.py, and the MonPoly .sig/.mfotl/.log files for the STAC, building, IoT and timed studies.